

Kafka vs RabbitMQ en una Red Social

Resumen: Este documento describe las diferencias entre Kafka y RabbitMQ y recomendaciones sobre su uso en un proyecto de red social.

1. Kafka – Para eventos y streaming de datos

Kafka es ideal para escenarios con muchos eventos generados por usuarios o sistemas que necesitan ser procesados en tiempo real o casi en tiempo real. Además, permite releer el historial de eventos.

Casos de uso en una red social:

Funcionalidad / Microservicio	Justificación
Feed de noticias / timeline	Publicaciones, likes y comentarios generan eventos continuos. Kafka permite que microservicios que construyen feeds lean eventos de manera eficiente.
Actividad de usuario (analytics)	Clicks, vistas de posts, interacciones. Kafka permite recolectar millones de eventos para métricas, estadísticas y machine learning.
Notificaciones en tiempo real	Publicaciones, menciones, comentarios. Kafka transmite eventos a microservicios de notificaciones.
Procesos de recomendación (ML)	Kafka permite enviar eventos al microservicio de recomendaciones sin bloquear la experiencia de usuario.
Logs de auditoría / seguridad	Registrar todas las acciones de usuario o cambios importantes. Kafka asegura persistencia y relectura de eventos.

Ventajas: - Altísimo throughput, ideal para millones de usuarios. - Relectura de eventos históricos. - Escalabilidad horizontal para muchos productores y consumidores.

2. RabbitMQ – Para tareas y mensajería confiable

RabbitMQ es ideal para trabajos específicos y colas de tareas, donde se requiere entrega garantizada y control de reintentos.

Casos de uso en una red social:

Funcionalidad / Microservicio	Justificación
Registro de usuario / validaciones	Enviar correo de verificación después de registrarse, garantizando ejecución aunque el servicio de correo esté caído.

Funcionalidad / Microservicio	Justificación
Envío de emails / SMS / push notifications	Maneja retries, dead-letter queues y aseguramiento de entrega.
Procesamiento de imágenes / videos	Redimensionar fotos, generar thumbnails o compresión mediante workers en cola.
Jobs programados o batch processing	Limpieza de datos antiguos, notificaciones programadas, reportes.
Eventos críticos de negocio	Bloqueo de usuarios, cambios en roles, pagos; requiere entrega garantizada.

Ventajas: - Garantía de entrega con ACK y retries. - Routing flexible (direct, topic, fanout). - Ideal para tareas que no requieren streaming masivo.

3. Arquitectura combinada sugerida

Combinar Kafka y RabbitMQ según el patrón:

```
[Microservicio Posts] --(Kafka: new post)--> [Feed Service] --> [Timeline Service]
[Microservicio Likes] --(Kafka: new like)--> [Analytics Service]

[User Service] --(RabbitMQ: send verification email)--> [Email Service]
[Media Service] --(RabbitMQ: process image/video)--> [Worker Pool]
```

✅ Kafka: eventos masivos y streaming en tiempo real. ✅ RabbitMQ: tareas confiables, colas de trabajo y mensajería crítica.